Reviewer Pack — Minimal Geometric Entropy Correlation with Frege Proof Size …

Entry 235c1debb572 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 06:02:59 UTC

Minimal Geometric Entropy Correlation with Frege Proof Size via Topological Degree

Entry ID: `235c1debb572` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 06:02:59 UTC

1. Conjecture

Field A (mathematical branch): Topological Degree Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

> For every d-regular graph G, the topological degree (td(G)) of its associated Tseitin formula φ_G is linearly correlated with its Frege proof size f(φ_G), such that td(G) = Θ(f(φ_G)).

Rationale (proposer's reasoning):

> Topological degree theory provides a geometric measure that captures the connectivity of spaces. If this topological property correlates with the complexity of proving tautologies, it could provide insight into the inherent difficulty of certain computational tasks.

Taxonomy category: `topological_degree` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2ba5abb0fe7441d4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every d-regular graph G, if the absolute difference between the topological degree (td(G)) and the Frege proof size (f(φ_G)) is less than or equal to 2 over at least 80% of the seeds, it supports the conjecture. Falsification occurs if any seed results in an absolute difference greater than 5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `topological degree AND Frege proof complexity` - `Tseitin formula AND linear correlation topological degree Frege proof size` - `degree of d-regular graph AND related to Frege proof length`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_d_regular_graph(d, n):
        graph = [[] for _ in range(n)]
        nodes = list(range(n))
        random.shuffle(nodes)

        for i in range(n):
            for j in range(i + 1, n):
                if len(graph[i]) < d and len(graph[j]) < d:
                    graph[i].append(j)
                    graph[j].append(i)

        return graph

    def topological_degree(graph):
        degrees = [len(neighbors) for neighbors in graph]
        return sum(degrees) / len(degrees)

    def frege_proof_size(n):
        # Simplified model: Frege proof size is proportional to n^2
```

```
        return n * n

    n = 10  # Start with a small n and increase
    td_values = []
    fgs_values = []

    while len(td_values) < 30:
        graph = generate_d_regular_graph(3, n)
        td = topological_degree(graph)
        fg = frege_proof_size(n)

        if td is not None and fg is not None:
            td_values.append(td)
            fgs_values.append(fg)

        n += 1

    correlation_coefficient = sum((td - mean_td) * (fg - mean_fg) for td, fg in zip(td_values, fgs
    mean_td = sum(td_values) / len(td_values)
    mean_fg = sum(fgs_values) / len(fgs_values)

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(td_values),
        "n_max": n - 1,
        "conjecture_holds": abs(correlation_coefficient) >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 7
    else:
        seeds = [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(abs(r["metric_value"]) > 5 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if abs(result["metr
        print(f"RESULT: FALSIFIED counterexample=\"large_difference\" first_failing_seed={first_fa
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e0e6ff9.py", line 86, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e0e6ff9.py", line 65, in run_trial
    correlation_coefficient = sum((td - mean_td) * (fg - mean_fg) for td, fg in zip(td_values, fgs_value
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e0e6ff9.py", line 65, in <genexpr>
    correlation_coefficient = sum((td - mean_td) * (fg - mean_fg) for td, fg in zip(td_values, fgs_value
                                           ^^^^^^^
NameError: cannot access free variable 'mean_td' where it is not associated with a value in enclosing s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the pre-registered support condition is met. | next: Investigate and fix the crash in the test code to ensure it can complete without errors.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17873
2	propose	ollama_remote	glm4:latest	0	0	20231
3	preregistration	ollama_remote	glm4:latest	0	0	9445
4	novelty	ollama_remote	glm4:latest	0	0	13846
5	novelty	ollama_remote	glm4:latest	0	0	25918
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19509
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18183
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12069
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17567
10	judge	ollama_remote	glm4:latest	0	0	9598

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 164239 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/235c1debb572.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/235c1debb572.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/235c1debb572.tar.gz` (if generated)